DSA Assignment

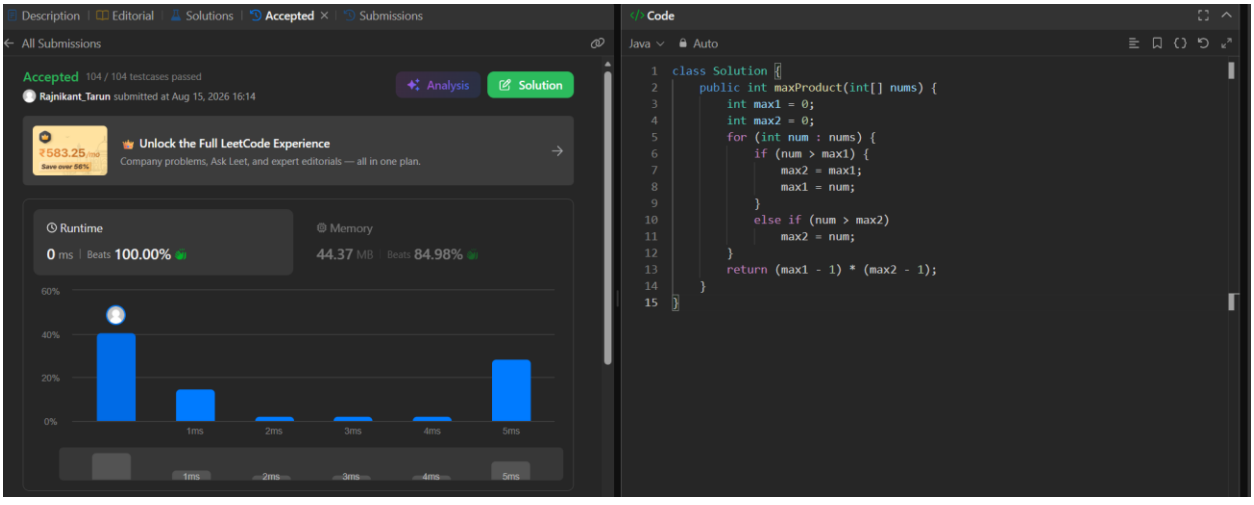
NAME – Rajnikant Kumar Tarun

Uni. Roll no. - 12515001275

Class Roll no. - 55

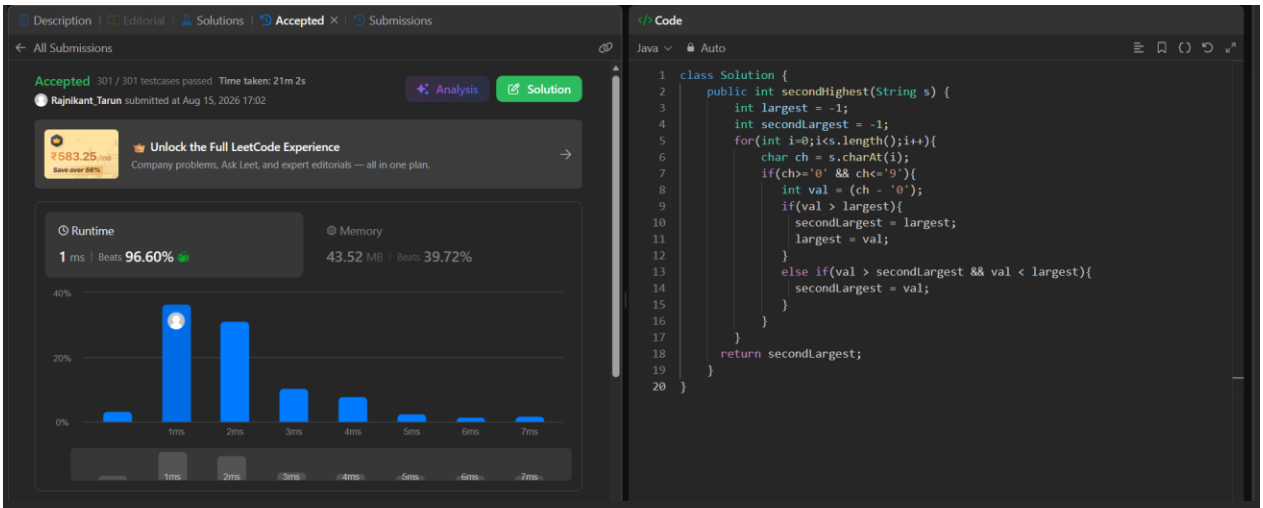
Section – FB

Q1. 1464, Maximum Product of Two Elements in an Array



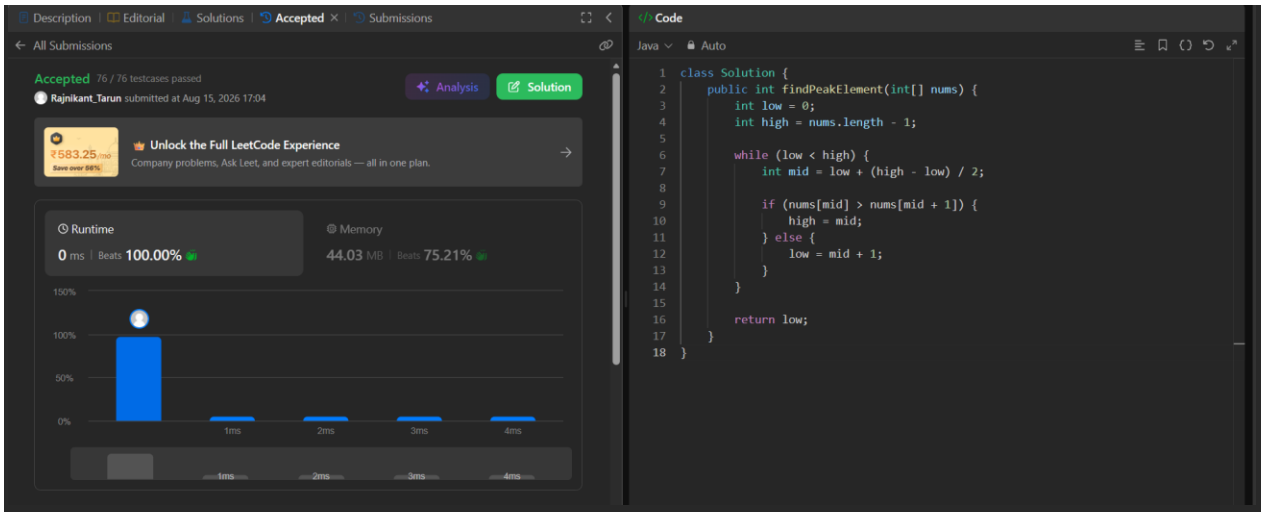
```java
class Solution {
    public int maxProduct(int[] nums) {
        int max1 = 0;
        int max2 = 0;
        for (int num : nums) {
            if (num > max1) {
                max2 = max1;
                max1 = num;
            }
            else if (num > max2)
                max2 = num;
        }
        return (max1 - 1) * (max2 - 1);
    }
}
```

Q.2 Second-Largest




```java
class Solution {
    public int secondHighest(String s) {
        int largest = -1;
        int secondLargest = -1;
        for(int i=0;i<s.length();i++){
            char ch = s.charAt(i);
            if(ch>='0' && ch<='9'){
               int val = (ch - '0');
               if(val > largest){
                 secondLargest = largest;
                 largest = val;
               }
               else if(val > secondLargest && val < largest){
                 secondLargest = val;
               }
            }
        }
      return secondLargest;
    }
}
```

Q3. Find Peak Element (162)



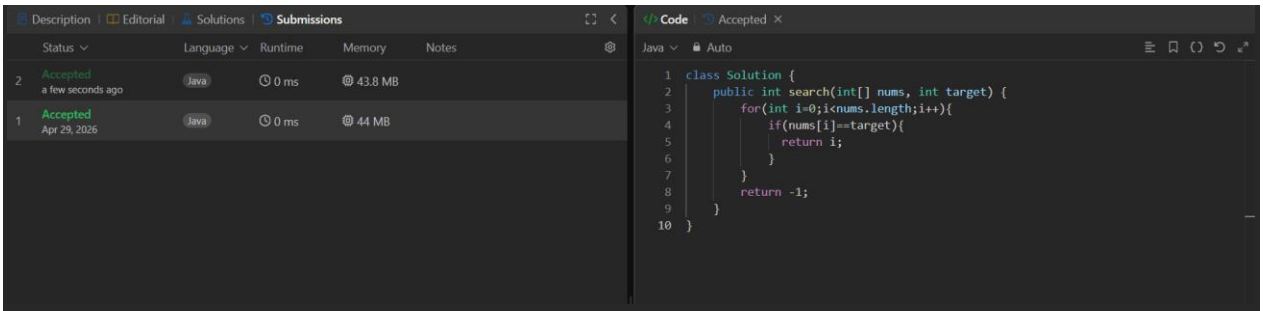

```java
class Solution {
    public int findPeakElement(int[] nums) {
        int low = 0;
        int high = nums.length - 1;

        while (low < high) {
            int mid = low + (high - low) / 2;

            if (nums[mid] > nums[mid + 1]) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }

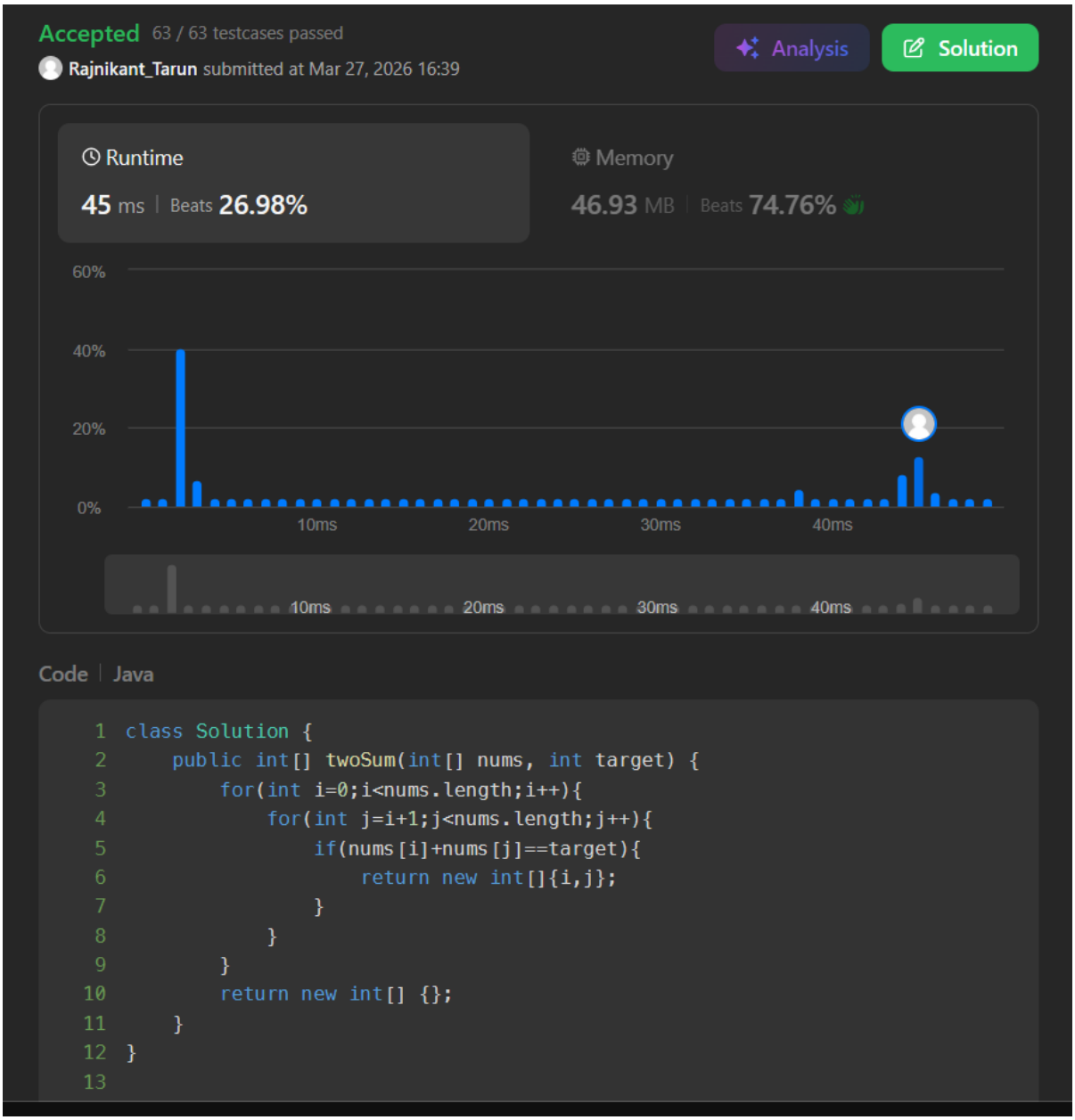
        return low;
    }
}
```

Q4. Search in Rotated Sorted Array (33)




```java
class Solution {
    public int search(int[] nums, int target) {
        for(int i=0;i<nums.length;i++){
            if(nums[i]==target){
              return i;
            }
        }
        return -1;
    }
}
```

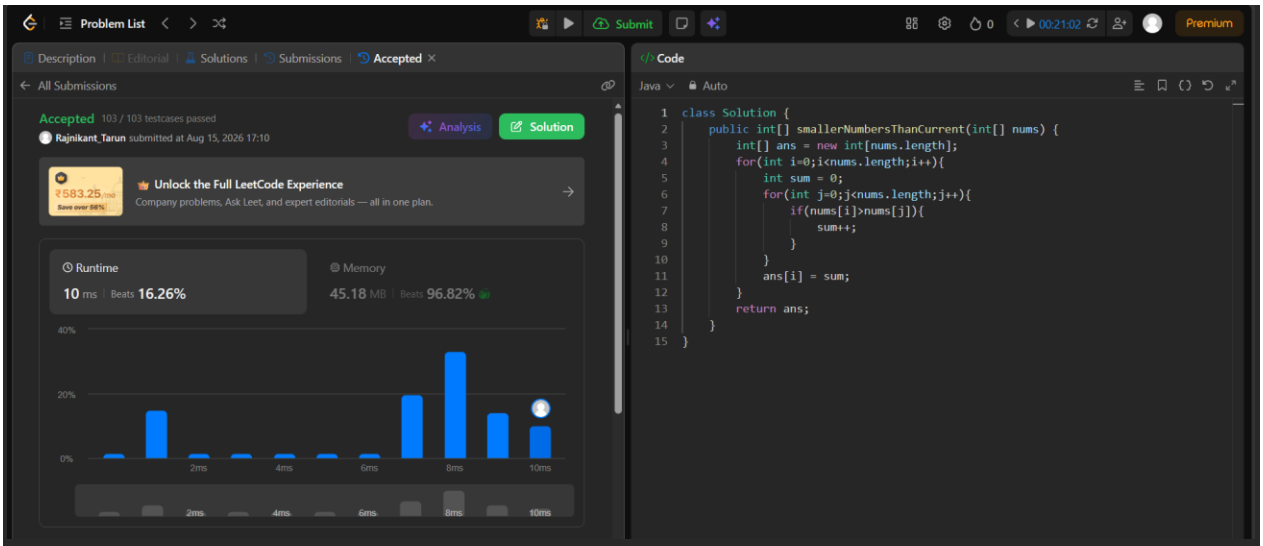
Q.5 Two Sum




```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        for(int i=0;i<nums.length;i++){
            for(int j=i+1;j<nums.length;j++){
                if(nums[i]+nums[j]==target){
                    return new int[]{i,j};
                }
            }
        }
```

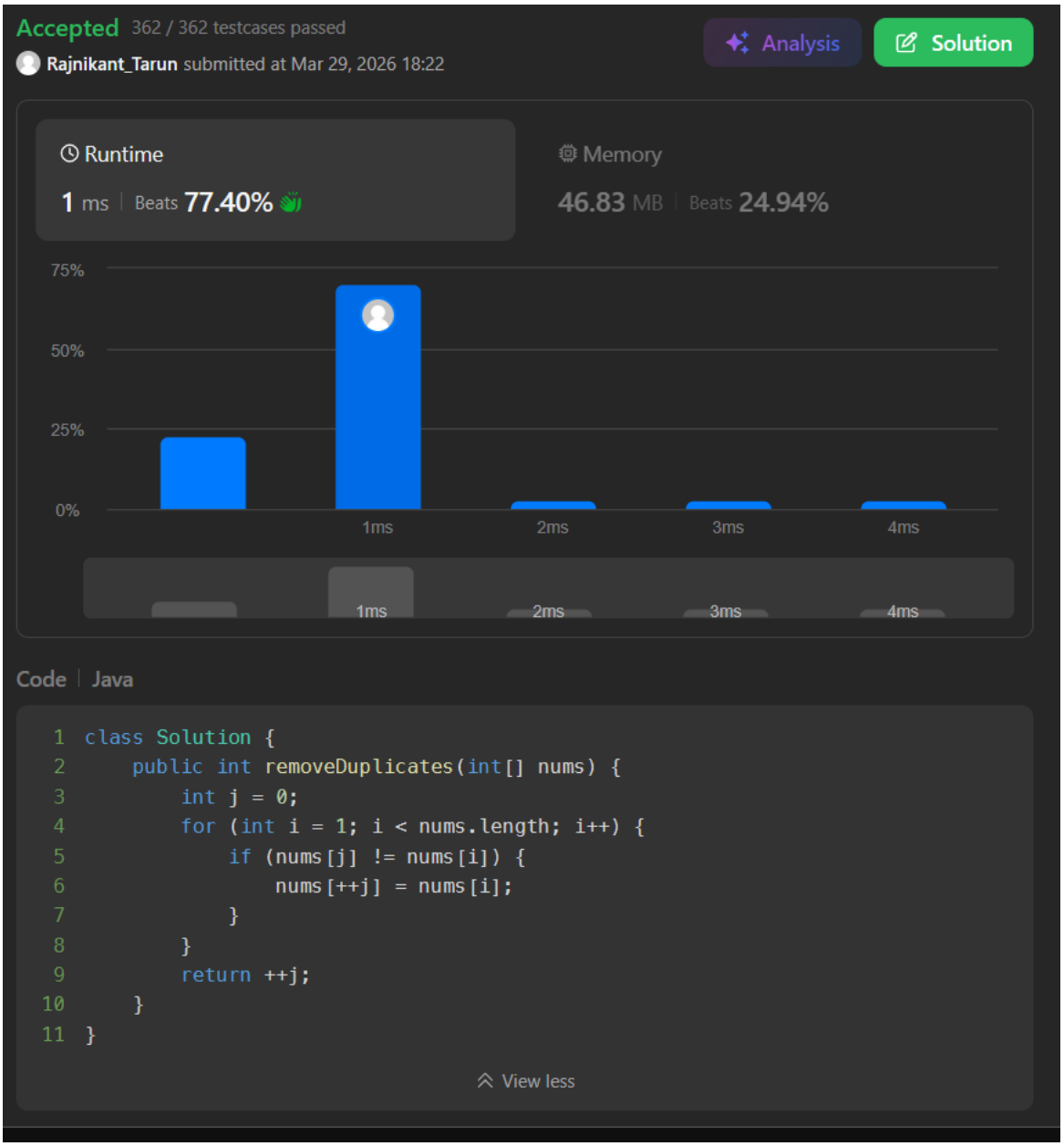
```
        return new int[] {};
    }
}
```

Q6. How Many Numbers Are Smaller Than the Current Number




```java
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {
        int[] ans = new int[nums.length];
        for(int i=0;i<nums.length;i++){
            int sum = 0;
            for(int j=0;j<nums.length;j++){
                if(nums[i]>nums[j]){
                    sum++;
                }
            }
            ans[i] = sum;
        }
        return ans;
    }
}
```

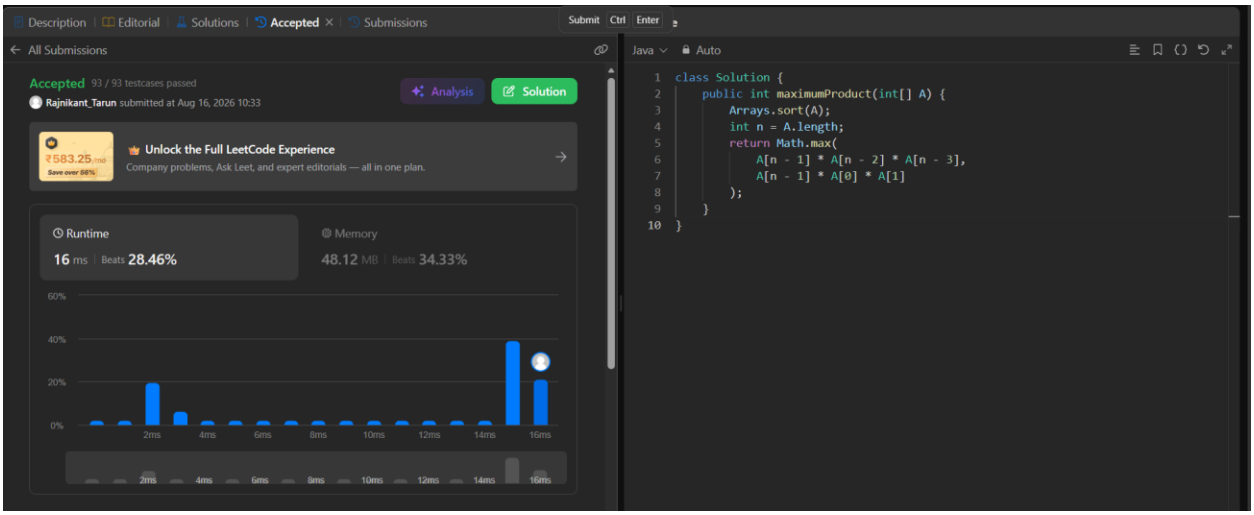
Q7. Remove Duplicates from Sorted Array




```java
class Solution {
    public int removeDuplicates(int[] nums) {
        int j = 0;
        for (int i = 1; i < nums.length; i++) {
            if (nums[j] != nums[i]) {
                nums[++j] = nums[i];
            }
        }
        return ++j;
    }
```

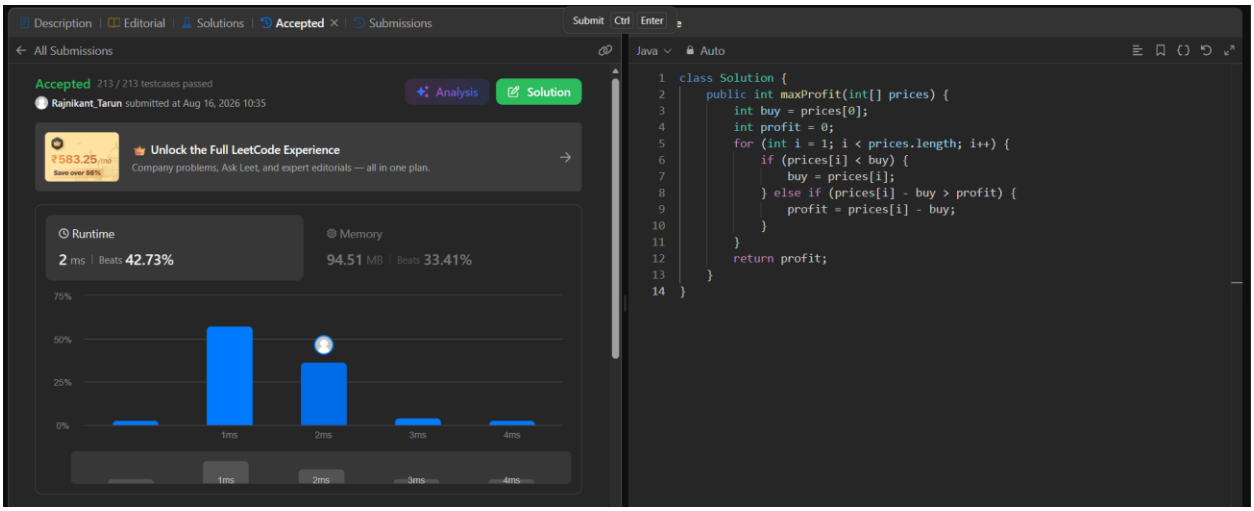
```
}
```

Q8. Maximum Product of Three Numbers



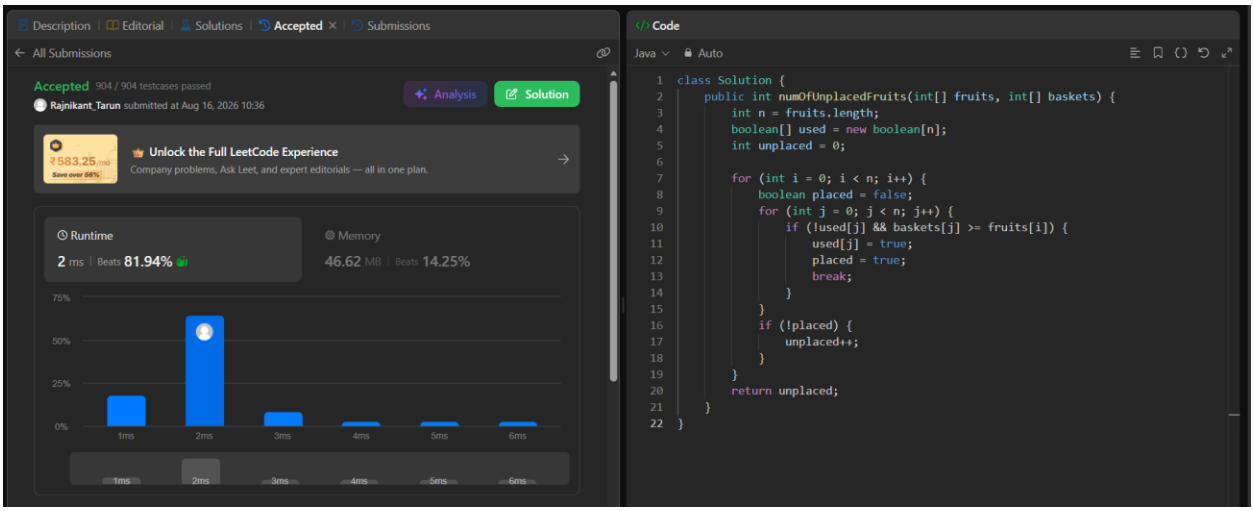

```java
class Solution {
    public int maximumProduct(int[] A) {
        Arrays.sort(A);
        int n = A.length;
        return Math.max(
            A[n - 1] * A[n - 2] * A[n - 3],
            A[n - 1] * A[0] * A[1]
        );
    }
}
```

Q9. Best Time to Buy and Sell Stock




```java
class Solution {
    public int maxProfit(int[] prices) {
        int buy = prices[0];
        int profit = 0;
        for (int i = 1; i < prices.length; i++) {
            if (prices[i] < buy) {
                buy = prices[i];
            } else if (prices[i] - buy > profit) {
                profit = prices[i] - buy;
            }
        }
        return profit;
    }
}
```

Q10. Fruits Into Baskets II




```java
class Solution {
    public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
        int n = fruits.length;
        boolean[] used = new boolean[n];
        int unplaced = 0;

        for (int i = 0; i < n; i++) {
            boolean placed = false;
            for (int j = 0; j < n; j++) {
                if (!used[j] && baskets[j] >= fruits[i]) {
                    used[j] = true;
                    placed = true;
                    break;
                }
            }
            if (!placed) {
                unplaced++;
            }
        }
        return unplaced;
    }
}
```